

# Document Database

 [systemdesignschool.io/fundamentals/document-database](https://systemdesignschool.io/fundamentals/document-database)



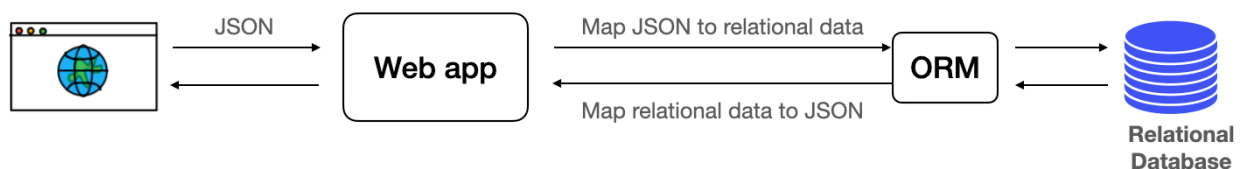
Key-value databases provide simple lookups. But many applications need nested data structures: user profiles with addresses, blog posts with comments, products with specifications. Document databases solve this problem.

## The ORM Problem

Douglas Crockford published the JSON specification in 2001. JSON became the standard for web APIs. Applications communicate using JSON. Browsers send JSON. Servers respond with JSON. But databases store tables and rows.

Storing JSON in SQL databases creates friction. A user profile contains nested addresses and multiple phone numbers. SQL requires separate tables: users, addresses, phone\_numbers. Insert one profile? Execute three INSERT statements. Read one profile? JOIN three tables.

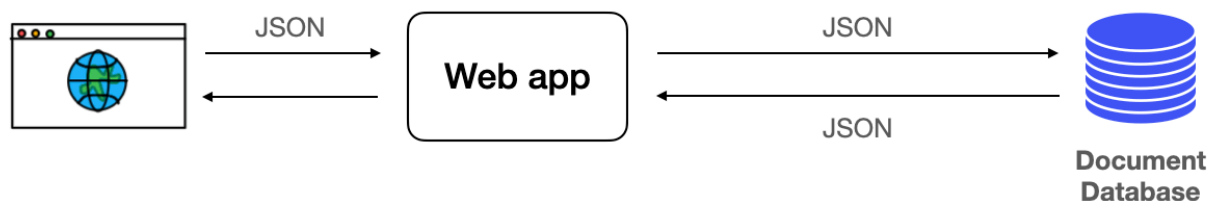
[Object-relational mappers](#) translate between JSON and SQL. ORMs work until they don't. Nested arrays require complex mapping. Schema migrations break existing code. Performance suffers from N+1 query problems.



## How Document Databases Work

---

Document databases store JSON directly. No translation layer. No ORM complexity. The application sends JSON. The database stores JSON. Reading returns JSON.



Each document is self-contained. A user profile includes all related data in one document. Addresses nest inside. Phone numbers nest inside. One read operation retrieves everything.

Documents can have different fields. One user has verified badges. Another doesn't. The database accepts both. No schema migration needed. Add fields to new documents. Old documents remain unchanged.

## When to Use Document Databases

---

Document databases work best for self-contained content without strong relationships. User profiles fit perfectly. A Reddit profile displays when someone visits your page. No complex queries needed. New features require new fields. Add **rewards:** `[]` without migration. The same applies to merchant profiles in Yelp, venue profiles in TicketMaster, and e-commerce listings.

Content management systems benefit from document databases. WordPress stores blog posts. Each post contains title, body text, and tags array. Posts are mostly independent. Retrieving one post shouldn't require joins. Flexible schemas allow custom fields per post.

Product catalogs match document patterns. An electronics product has different specifications than clothing. Electronics need screen size and battery life. Clothing needs size and material. Document databases store both in the same collection with different fields.

## When Not to Use Document Databases

---

Avoid document databases when data needs strong consistency, complex relationships, and strict [normalization](#). Banking applications managing account transfers require ACID transactions. Use relational databases instead.

## Interactive Example

---

[Try running MongoDB commands](#) to see document databases in action:

```
// create a new collection db.createCollection("users"); // insert one document
into the collection db.users.insert({ name: "John Doe", age: 30, email:
"john.doe@example.com" }); // insert two documents into the collection
db.users.insertMany([ { name: "Jane Smith", age: 28, email:
"jane.smith@example.com" }, { name: "Bob Johnson", age: 45, email:
"bob.johnson@example.com" }, ]); // find all documents db.users.find(); // find
a specific document db.users.findOne({ name: "John Doe" }); // update a document
db.users.updateOne({ name: "John Doe" }, { $set: { age: 31 } }); // delete a
document db.users.deleteOne({ name: "John Doe" }); // add a new field
db.users.updateOne( { name: "John Doe" }, { $set: { address: "123 Main St,
Springfield" } } );
```